

Prompt 1 — Fix Overbooking en Creación de Reservas + Camaje Clickable en Planning

PARTE 1 — Fix overbooking: POST /api/reservations no valida solapamiento

El bug

El endpoint `POST /api/reservations` (línea ~38714) crea la reserva directamente **sin chequear si la habitación ya tiene una reserva en esas fechas**. El método `checkOverbooking` existe en `storage` pero nunca se llama desde este endpoint. Por eso: si hay una reserva del 11/03 al 13/03 en la hab 405, se puede crear otra del 10/03 al 12/03 en la misma hab porque las fechas "envuelven" la existente.

Fix en `server/routes.ts`

En `app.post("/api/reservations", ...)`, agregar la validación de overbooking ANTES de llamar a `storage.createReservation`. Insertar este bloque después de la validación de fechas y antes del `storage.createReservation`:

```
app.post("/api/reservations", async (req, res) => {
  try {
    // ... sanitización de campos existente sin cambios ...

    const data = {
      ...req.body,
      reservationCode: req.body.reservationCode || storage.generateReservationCod
      createdAt: req.body.createdAt ? new Date(req.body.createdAt) : new Date(),
    };

    // ✅ AGREGAR: Validación de overbooking antes de crear
    if (data.roomId && data.checkInDate && data.checkOutDate) {
      const hasConflict = await storage.checkOverbooking(
        data.roomId,
        data.checkInDate,
        data.checkOutDate
      );
      if (hasConflict) {
```

```

    const room = await storage.getRoom(data.roomId);
    return res.status(409).json({
      error: `La habitación ${room?.roomNumber || data.roomId} ya tiene una r
    });
  }
}

const reservation = await storage.createReservation(data);
// ... resto sin cambios ...

```

Fix también en PATCH /api/reservations/:id

El PATCH actual solo valida overbooking si cambia el `roomId`. Hay que completarlo para que también valide cuando cambian las fechas sin cambiar de habitación:

```

// En PATCH /api/reservations/:id, reemplazar el bloque de validación existente:

// Determinar roomId, checkIn y checkOut finales
const finalRoomId = req.body.roomId || existing.roomId;
const finalCheckIn = req.body.checkInDate || existing.checkInDate;
const finalCheckOut = req.body.checkOutDate || existing.checkOutDate;

// Validar si cambió el cuarto O las fechas
const roomChanged = req.body.roomId && req.body.roomId !== existing.roomId;
const datesChanged = req.body.checkInDate || req.body.checkOutDate;

if (roomChanged || datesChanged) {
  const hasConflict = await storage.checkOverbooking(
    finalRoomId,
    finalCheckIn,
    finalCheckOut,
    req.params.id // excluir la reserva actual
  );
  if (hasConflict) {
    const room = await storage.getRoom(finalRoomId);
    return res.status(409).json({
      error: `La habitación ${room?.roomNumber || finalRoomId} ya tiene una reser
    });
  }
}

```

Fix en frontend: mostrar el error correctamente

En `ReservationFormDialog` y `QuickReservationDialog`, en el `onError` de `createReservationMutation`:

```

onError: (error: any) => {
  const message = error?.data?.error || error?.message || "No se pudo crear la re
  toast({
    title: "Conflicto de fechas",
    description: message,
    variant: "destructive",
  });
},

```

PARTE 2 — Click en camaje (columna izquierda del planning) para editar

El objetivo

En la columna izquierda del planning, cada habitación muestra número, tipo y camaje. Al hacer click en el camaje, abrir un popover/dialog para cambiarlo.

2a — Agregar estado en `planning.tsx`

```

const [editingBedConfig, setEditingBedConfig] = useState<{ roomId: string; roomNu

```

2b — Mutation para actualizar `bedConfig`

```

const updateBedConfigMutation = useMutation({
  mutationFn: async ({ roomId, bedConfig }: { roomId: string; bedConfig: string }
    const res = await apiRequest("PATCH", `/api/rooms/${roomId}`, { bedConfig });
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/planning"] });
    queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
    setEditingBedConfig(null);
    toast({ title: "Camaje actualizado" });
  },
  onError: () => {
    toast({ title: "Error al actualizar camaje", variant: "destructive" });
  },
});

```

2c — Hacer el camaje clickeable en la columna izquierda

Buscar el bloque que muestra `room.bedConfig` en la columna sticky izquierda (cerca de `data-testid={room-header-`${room.id}`} `):`

```
// Reemplazar:
{room.bedConfig && (
  <span className="text-[10px] text-muted-foreground">{room.bedConfig}</span>
)}

// Por:
{room.bedConfig && (
  <span
    className="text-[10px] text-muted-foreground underline decoration-dotted cursor-pointer"
    onClick={e => {
      e.stopPropagation();
      setEditingBedConfig({ roomId: room.id, roomNumber: room.roomNumber, current: room.bedConfig });
    }}
    title="Click para cambiar camaje"
    data-testid={`button-edit-bedconfig-${room.id}`}
  >
    {bedConfigLabels[room.bedConfig] || room.bedConfig}
  </span>
)}

```

2d — Dialog de edición de camaje

Agregar al final del JSX del componente:

```
<Dialog open={!!editingBedConfig} onOpenChange={open => !open && setEditingBedConfig}>
  <DialogContent className="max-w-xs">
    <DialogHeader>
      <DialogTitle>Cambiar Camaje — Hab. {editingBedConfig?.roomNumber}</DialogTitle>
    </DialogHeader>
    <div className="py-4">
      <Label className="text-sm mb-2 block">Configuración de camas</Label>
      <Select
        value={editingBedConfig?.current || ""}
        onChange={value => setEditingBedConfig(prev => prev ? { ...prev, current: value } : { current: value })}
      >
        <SelectTrigger data-testid="select-bedconfig">
          <SelectValue placeholder="Seleccionar" />
        </SelectTrigger>
        <SelectContent>
          {
            [
              { value: "MAT", label: "Matrimonial" },
              { value: "TWIN", label: "Twin (2 camas)" },
            ]
          }
        </SelectContent>
      </Select>
    </div>
  </DialogContent>
</Dialog>

```

```

        { value: "MAT_CC", label: "Matrimonial + Cama cuna" },
        { value: "TWIN_CC", label: "Twin + Cama cuna" },
        { value: "MAT_EXTRA", label: "Matrimonial + Extra" },
        { value: "MAT_CC_EXTRA", label: "Matrimonial + Cuna + Extra" },
    ].map(opt => (
        <SelectItem key={opt.value} value={opt.value}>{opt.label}</SelectItem
    )))
</SelectContent>
</Select>
</div>
<DialogFooter>
    <Button variant="outline" onClick={() => setEditingBedConfig(null)}>Cancela
    <Button
        onClick={() => {
            if (editingBedConfig) {
                updateBedConfigMutation.mutate({ roomId: editingBedConfig.roomId, bed
            }
        }}
        disabled={updateBedConfigMutation.isPending}
        data-testid="button-confirm-bedconfig"
    >
        {updateBedConfigMutation.isPending ? "Guardando..." : "Guardar"}
    </Button>
</DialogFooter>
</DialogContent>
</Dialog>

```

Resumen de archivos modificados

Archivo	Cambio
server/routes.ts	POST /api/reservations → agregar checkOverbooking; PATCH → validar también cambios de fecha
client/src/pages/planning.tsx	Estado editingBedConfig , mutation, span clickeable, Dialog de camaje